C++ Object Copying: Shallow vs Deep, Constructors & Assignments

Basic Copying

```
class Stud {
public:
    int age;
    string name;
};
```

now if we write `Stud s2 = s1;` then each member is copied:

- `s2.age = s1.age`
- `s2.name = s1.name`

Pointer Copying & Shallow Copy

```
class Stud {
public:
    int *marks;
    Stud(int m) {
        marks = new int(m);
    }
};
```

now if we do `s2 = s1` then both contains pointers, pointing to same address.

```
Stud s1(90);
s2 = s1;
*s2.marks = 99;
```

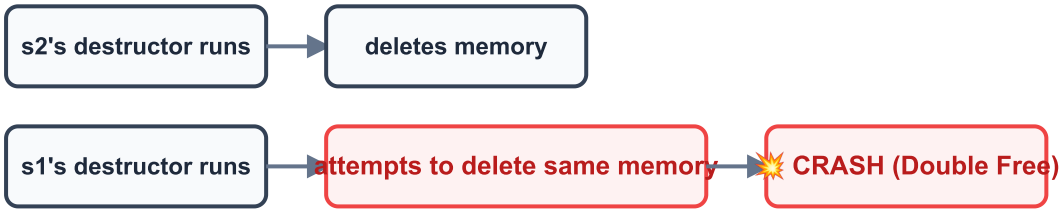
this will change marks for both `s1` and `s2` as address is common. This is **[Shallow Copying]**.

The Dangers of Shallow Copying

This shallow copying is dangerous because updating / deleting one updates/deletes other too. Eg let:

```
~Stud() {
    delete marks;
}
```

now if `s2 = s1` and when the objects are destroyed it leads to a **Double deletion** of same address.



`Stud { int age; string name; }`
`Stud s1(20, Aadya); s2 = s1;`
compiler generated copy constructor does member wise copying

If a class has pointer address, then pointer address is copied and not pointed to memory.

Shallow vs Deep Copy

Shallow copy	deep copy
copies pointer address and member values	copies the underlying resources into a separate memory, instead of copying pointer address it copies pointed to values
both object may share some memory	each object have independent memory
modifying 1 affects other	modifying 1 doesnt affect other
compiler generated copy constructors generally follow this rule	usually requires custom copy constructors and copy assignments

Copy Constructor Implementation

A special constructor that creates a new object using an existing object of same class.

Syntax:

```
className (const className &obj);
```

Eg

```
class Student {
public:
    int age;
    Student(int a) {
        age = a;
    }
    Student(const Student &obj) {
        age = obj.age;
    }
    // const prevents modifying orig object
};
```

```
Student s1(20);
Student s2 = s1;
```

`s1` is passed by reference and its data is copied into new object `s2`.

> **Q: What if we use `Student(Student obj)`?**
> Then it requires passing object by value. Passing by value needs copying `obj` which will call copy constructor again (hence **∞ loop**). That why we pass by reference.

Copy Constructor vs Copy Assignment Operator

```
Stud s1(20);
Stud s2 = s1; // copy constructor

Stud s3(30);
s3 = s1;      // copy assignment operator
```

Copy constructor	Copy assignment operator
1) creates a new object	works on already existing object assignment
2) initialization	replaces existing data with new data
3) `Stud(const Stud &obj)`	`Stud & operator=(const Stud &obj)`

Preventing Copying

- If we dont write copy constructor C++ can generate implicit copy constructor for a class which does member wise copying.

> **How to prevent copying objects:**
>
> - If we make copy constructor of a class `private`, it is a way to prevent copying objects.
> - If we write `Stud(const Stud &obj) = delete;` this also prevents copying.

Copy Assignment Operator

```
Stud & operator=(const Stud &obj) {
    age = obj.age;
    return *this;
    // [this refers to destination obj]
} // -> inside the Stud class just as we made copy constructor
```

Keyword Breakdown:

- `Stud &` return a reference to destination object as it already exists.
- `this` is a pointer to current destination object.
- `*this` refers to current destination object itself.

Why we return `*this`:

imagine `s3 = s2 = s1;`

```
s3 = s2 = s1;

this is evaluated right-to-left as:

( s3 = ( s2 = s1 ) )
```

WARNING: default copy constructors and default copy assignment constructors have issues as they copy pointer addresses instead of duplicating allocated resources.